

PYTHON WEEK 3 TEST

Collections Framework: List, Dict, Set, Tuple

Instructions: (1) Solve all questions. (2) For print-output questions, write exact output. (3) For MCQ, choose one correct option.
(4) Total: 100 questions | L1: 30 | L2: 70 | Output: 80 | MCQ: 20.

Level 1

Part A: Predict the Output

Q1. Predict the output for sort vs sorted.

Code Snippet

```
nums = [9, 8, 3, -8, -12]
out = sorted(nums, key=abs)
nums.sort(reverse=True)
print(out)
print(nums)
```

Answer:

Q2. Predict the output for set comprehension.

Code Snippet

```
s = {x * x for x in range(1, 9) if x % 2 == 0}
print(s)
```

Answer:

Q3. Predict the output for reversed iteration.

Code Snippet

```
tokens = ['green', 'black', 'cyan', 'red']
print(list(reversed(tokens)))
tokens.reverse()
print(tokens)
```

Answer:

Q4. Predict the output of this slicing example.

Code Snippet

```
nums = [1, 12, 17, 1, 23, 2]
print(nums[1:4])
```

Answer:

Q5. Predict the output for remove/discard handling.

Code Snippet

```
s = {10, 20, 30}
try:
    s.remove(109)
except KeyError:
    print('keyerror')
s.discard(109)
print(s)
```

Answer:

Q6. Predict the output for pop and remaining dictionary.

Code Snippet

```
d = {'p': 10, 'q': 20, 'r': 30}
print(d.pop('q'))
print(d)
```

Answer:

Q7. Predict the output of this slicing example.

Code Snippet

```
nums = [5, 21, 6, 28, 3, 9]
print(nums[1:4])
```

Answer:

Q8. Predict the output for get with default values.

Code Snippet

```
profile = {'name': 'Ana', 'score': 88}
print(profile.get('score'))
print(profile.get('city', 'NA'))
```

Answer:

Q9. Predict the output of this slicing example.

Code Snippet

```
nums = [22, 3, 6, 10, 11, 13]
print(nums[2:6])
```

Answer:

Q10. Predict the output for slicing and reverse slicing.

Code Snippet

```
t = (8, 8, 3, 3, 2)
print(t[1:4])
print(t[::-1])
```

Answer:

Q11. Predict the output of this slicing example.

Code Snippet

```
nums = [7, 30, 20, 7, 30, 1]
print(nums[2:4])
```

Answer:

Q12. Predict the output after insert and pop operations.

Code Snippet

```
arr = [2, 2, 2, 1, 9]
arr.insert(1, 75)
print(arr.pop(2))
print(arr)
```

Answer:

Q13. Predict the output for.setdefault behavior.

Code Snippet

```
d = {'a': 1}
print(d.setdefault('a', 99))
print(d.setdefault('b', 50))
print(d)
```

Answer:

Q14. Predict the output for shallow copy with nested list.

Code Snippet

```
d1 = {'scores': [10, 20]}
d2 = d1.copy()
d2['scores'].append(30)
print(d1)
print(d2)
```

Answer:

Q15. Predict the output after insert and pop operations.

Code Snippet

```
arr = [1, 9, 6, 3, 7]
arr.insert(3, 69)
print(arr.pop(4))
print(arr)
```

Answer:

Q16. Predict the output for extended unpacking.

Code Snippet

```
t = (1, 2, 3, 4, 5)
first, *mid, last = t
print(first)
print(mid)
print(last)
```

Answer:

Part B: MCQ (Single Correct Option)

Q17. What does `lst.pop()` do by default?

- A) Removes first element
- B) Removes last element and returns it
- C) Removes all duplicates
- D) Returns index of max

Q18. For optional lookup, which is safest?

- A) `d[k]`
- B) `d.get(k, default)`
- C) `del d[k]`
- D) `d.popitem()`

Q19. Correct one-element tuple syntax is:

- A) `(5)`
- B) `[5,]`
- C) `(5,)`
- D) `{5}`

Q20. Set elements must be:

- A) Mutable
- B) Hashable
- C) Sorted
- D) Numeric

Q21. What does `d.copy()` create for nested mutable values?

- A) Deep copy
- B) Shallow copy
- C) Read-only proxy
- D) No copy

Q22. `dict.fromkeys(keys, [])` can be risky because:

- A) It is very slow
- B) It rejects list values
- C) Same list is shared across keys
- D) It sorts keys automatically

Q23. Which statement about `sorted(lst)` is correct?

- A) It mutates `lst` in-place
- B) It returns a new sorted list
- C) It returns `None`
- D) It works only on int lists

Q24. Which operation adds each element of another iterable to an existing list?

- A) append
- B) extend
- C) insert
- D) remove

Q25. Tuples are usually preferred when data is:

- A) frequently appended
- B) fixed-shape record
- C) key-value mapping
- D) unordered unique collection

Q26. What does `a[1:1]` return for a list?

- A) IndexError
- B) The element at index 1
- C) Empty list
- D) None

Q27. Which method removes an element without error if missing?

- A) remove
- B) discard
- C) pop
- D) clear

Q28. When `b = a` for lists, what is true?

- A) `b` is a deep copy
- B) `b` points to same object
- C) `b` is immutable copy
- D) `b` is tuple

Q29. In dictionaries, `in` checks membership of:

- A) Values
- B) Key-value tuples
- C) Keys
- D) Indices

Q30. Main strength of set in DSA problems:

- A) Indexing
- B) Stable order
- C) Fast membership checks
- D) Key-value mapping

Level 2

Part A: Predict the Output

Q31. Predict the output for defaultdict(int) counting.

Code Snippet

```
from collections import defaultdict
freq = defaultdict(int)
for ch in 'abcab':
    freq[ch] += 1
print(dict(freq))
```

Answer:

Q32. Predict the output for nested flattening with conditional selection.

Code Snippet

```
matrix = [[4, 8], [4, 8], [3, 6]]
flat = [x for row in matrix for x in row if x % 2 == 0]
print(flat)
print(sum(flat))
```

Answer:

Q33. Predict the output for dispatch table call.

Code Snippet

```
def add(a, b):
    return a + b

def sub(a, b):
    return a - b

ops = {'+': add, '-': sub}
print(ops['+'](8, 4), ops['-'](8, 4))
```

Answer:

Q34. Predict the output using Counter.

Code Snippet

```
from collections import Counter
c = Counter([2, 2, 3, 3, 3, 4])
print(c)
print(c.most_common(1))
```

Answer:

Q35. Predict the output for sorting by a tuple key.

Code Snippet

```
items = [('p', 58), ('q', 52), ('r', 87)]
print(sorted(items, key=lambda x: (x[1], x[0])))
```

Answer:

Q36. Predict the output for function return unpacking and starred target.

Code Snippet

```
def get_data():  
    return 6, 11, 14, 8  
first, *mid, last = get_data()  
print(first)  
print(mid)  
print(last)
```

Answer:

Q37. Predict the output for zip truncation and zip_longest padding.

Code Snippet

```
from itertools import zip_longest  
a = [14, 24, 34]  
b = ['x', 'y']  
print(list(zip(a, b)))  
print(list(zip_longest(a, b, fillvalue='NA')))
```

Answer:

Q38. Predict the output for sorting dict items by value.

Code Snippet

```
d = {'x': 6, 'y': 4, 'z': 5}  
out = sorted(d.items(), key=lambda kv: kv[1], reverse=True)  
print(out)
```

Answer:

Q39. Predict the output for dispatch table call.

Code Snippet

```
def add(a, b):  
    return a + b  
  
def sub(a, b):  
    return a - b  
ops = {'+': add, '-': sub}  
print(ops['+'](10, 3), ops['-'](10, 3))
```

Answer:

Q40. Predict the output for MappingProxyType view.

Code Snippet

```
from types import MappingProxyType
base = {'a': 6}
view = MappingProxyType(base)
base['a'] = 14
print(view['a'])
```

Answer:

Q41. Predict the output for reversed/sorted behavior and tuple identity slice.

Code Snippet

```
t = (8, 4, 6)
print(tuple(reversed(t)))
print(sorted(t))
print(t[:] is t)
```

Answer:

Q42. Predict the output for MappingProxyType view.

Code Snippet

```
from types import MappingProxyType
base = {'a': 1}
view = MappingProxyType(base)
base['a'] = 9
print(view['a'])
```

Answer:

Q43. Predict the output for relation operators on sets.

Code Snippet

```
a = {3, 4}
b = {3, 4, 5}
print(a <= b, a < b)
print(b >= a, b > a)
```

Answer:

Q44. Predict the output for nested-loop set comprehension.

Code Snippet

```
matrix = [[3, 4], [4, 5], [5, 6]]
flat = {x for row in matrix for x in row}
print(flat)
```

Answer:

Q45. Predict the output with operator precedence in set expressions.

Code Snippet

```
a = {4, 5, 6}
b = {5, 6, 7}
c = {6, 7, 8}
print(a | b & c)
print((a | b) & c)
```

Answer:

Q46. Predict the output for nested flattening with conditional selection.

Code Snippet

```
matrix = [[3, 8], [4, 8], [3, 6]]
flat = [x for row in matrix for x in row if x % 2 == 0]
print(flat)
print(sum(flat))
```

Answer:

Q47. Predict the output for combined filtering and indexed transformation.

Code Snippet

```
nums = [6, 7, 9, 1, 9, 9]
out = [(i, x * x) for i, x in enumerate(nums) if x % 2 == 1]
print(out[-2:])
```

Answer:

Q48. Predict the output for dispatch table call.

Code Snippet

```
def add(a, b):  
    return a + b  
  
def sub(a, b):  
    return a - b  
ops = {'+': add, '-': sub}  
print(ops['+'](13, 2), ops['-'](13, 2))
```

Answer:

Q49. Predict the output for matrix creation pitfall.

Code Snippet

```
grid = [[0] * 3] * 2  
grid[0][1] = 9  
print(grid)
```

Answer:

Q50. Predict the output for combined filtering and indexed transformation.

Code Snippet

```
nums = [5, 1, 10, 12, 6, 7]  
out = [(i, x * x) for i, x in enumerate(nums) if x % 2 == 1]  
print(out[-2:])
```

Answer:

Q51. Predict the output for relation operators on sets.

Code Snippet

```
a = {4, 5}  
b = {4, 5, 6}  
print(a <= b, a < b)  
print(b >= a, b > a)
```

Answer:

Q52. Predict the output for first-duplicate detection.

Code Snippet

```
nums = [6, 2, 4, 3, 4, 5, 3]
seen = set()
first_dup = None
for x in nums:
    if x in seen:
        first_dup = x
        break
    seen.add(x)
print(first_dup)
```

Answer:

Q53. Predict the output for relation operators on sets.

Code Snippet

```
a = {6, 7}
b = {6, 7, 8}
print(a <= b, a < b)
print(b >= a, b > a)
```

Answer:

Q54. Predict the output with operator precedence in set expressions.

Code Snippet

```
a = {2, 3, 4}
b = {3, 4, 5}
c = {4, 5, 6}
print(a | b & c)
print((a | b) & c)
```

Answer:

Q55. Predict the output for sorting by a tuple key.

Code Snippet

```
items = [('p', 82), ('q', 62), ('r', 77)]
print(sorted(items, key=lambda x: (x[1], x[0])))
```

Answer:

Q56. Predict the output for nested flattening with conditional selection.

Code Snippet

```
matrix = [[1, 7], [3, 8], [4, 8]]
flat = [x for row in matrix for x in row if x % 2 == 0]
print(flat)
print(sum(flat))
```

Answer:

Q57. Predict the output for defaultdict(int) counting.

Code Snippet

```
from collections import defaultdict
freq = defaultdict(int)
for ch in 'cabca':
    freq[ch] += 1
print(dict(freq))
```

Answer:

Q58. Predict the output for deep copy in nested list.

Code Snippet

```
import copy
src = [[1], [2]]
cloned = copy.deepcopy(src)
cloned[0].append(7)
print(src)
print(cloned)
```

Answer:

Q59. Predict the output using starred unpacking.

Code Snippet

```
nums = [13, 12, 6, 1, 15]
first, *mid, last = nums
print(first)
print(mid)
print(last)
```

Answer:

Q60. Predict the output using Counter.

Code Snippet

```
from collections import Counter
c = Counter([3, 3, 4, 4, 4, 5])
print(c)
print(c.most_common(1))
```

Answer:

Q61. Predict the output for combined filtering and indexed transformation.

Code Snippet

```
nums = [12, 2, 12, 1, 9, 6]
out = [(i, x * x) for i, x in enumerate(nums) if x % 2 == 1]
print(out[-2:])
```

Answer:

Q62. Predict the output for dict key collision of bool/int.

Code Snippet

```
d = {}
d[6] = 'int-like'
d[6.0] = 'float-like'
print(d)
print(len(d))
```

Answer:

Q63. Predict the output for first-duplicate detection.

Code Snippet

```
nums = [5, 1, 3, 2, 3, 4, 2]
seen = set()
first_dup = None
for x in nums:
    if x in seen:
        first_dup = x
        break
    seen.add(x)
print(first_dup)
```

Answer:

Q64. Predict the output for function return unpacking and starred target.

Code Snippet

```
def get_data():  
    return 7, 11, 14, 9  
first, *mid, last = get_data()  
print(first)  
print(mid)  
print(last)
```

Answer:

Q65. Predict the output for sorting by a tuple key.

Code Snippet

```
items = [('p', 75), ('q', 72), ('r', 58)]  
print(sorted(items, key=lambda x: (x[1], x[0])))
```

Answer:

Q66. Predict the output for function return unpacking and starred target.

Code Snippet

```
def get_data():  
    return 5, 10, 13, 7  
first, *mid, last = get_data()  
print(first)  
print(mid)  
print(last)
```

Answer:

Q67. Predict the output for nested flattening with conditional selection.

Code Snippet

```
matrix = [[3, 5], [2, 6], [1, 8]]  
flat = [x for row in matrix for x in row if x % 2 == 0]  
print(flat)  
print(sum(flat))
```

Answer:

Q68. Predict the output for combined filtering and indexed transformation.

Code Snippet

```
nums = [9, 11, 7, 11, 7, 2]
out = [(i, x * x) for i, x in enumerate(nums) if x % 2 == 1]
print(out[-2:])
```

Answer:

Q69. Predict the output with operator precedence in set expressions.

Code Snippet

```
a = {3, 4, 5}
b = {4, 5, 6}
c = {5, 6, 7}
print(a | b & c)
print((a | b) & c)
```

Answer:

Q70. Predict the output for in-place update return values.

Code Snippet

```
s = {1, 2, 3, 4}
print(s.difference_update({2, 4}))
print(s)
```

Answer:

Q71. Predict the output for ChainMap lookup order.

Code Snippet

```
from collections import ChainMap
cfg = ChainMap({'theme': 'solarized'}, {'theme': 'midnight', 'lang': 'en'})
print(cfg['theme'], cfg['lang'])
```

Answer:

Q72. Predict the output for sorting dict items by value.

Code Snippet

```
d = {'x': 4, 'y': 2, 'z': 3}
out = sorted(d.items(), key=lambda kv: kv[1], reverse=True)
print(out)
```

Answer:

Q73. Predict the output with operator precedence in set expressions.

Code Snippet

```
a = {1, 2, 3}
b = {2, 3, 4}
c = {3, 4, 5}
print(a | b & c)
print((a | b) & c)
```

Answer:

Q74. Predict the output for zip truncation and zip_longest padding.

Code Snippet

```
from itertools import zip_longest
a = [10, 20, 30]
b = ['x', 'y']
print(list(zip(a, b)))
print(list(zip_longest(a, b, fillvalue='NA')))
```

Answer:

Q75. Predict the output for nested-loop set comprehension.

Code Snippet

```
matrix = [[1, 2], [2, 3], [3, 4]]
flat = {x for row in matrix for x in row}
print(flat)
```

Answer:

Q76. Predict the output for dispatch table call.

Code Snippet

```
def add(a, b):
    return a + b

def sub(a, b):
    return a - b

ops = {'+': add, '-': sub}
print(ops['+'](11, 2), ops['-'](11, 2))
```

Answer:

Q77. Predict the output of next() with fallback after generator filtering.

Code Snippet

```
names = ['arya', 'neo', 'trinity', 'nora']
g = (x.upper() for x in names if x.startswith('n'))
print(next(g, 'NA'))
print(next(g, 'NA'))
print(next(g, 'NA'))
```

Answer:

Q78. Predict the output for MappingProxyType view.

Code Snippet

```
from types import MappingProxyType
base = {'a': 5}
view = MappingProxyType(base)
base['a'] = 13
print(view['a'])
```

Answer:

Q79. Predict the output for nested flattening with conditional selection.

Code Snippet

```
matrix = [[2, 6], [3, 6], [1, 8]]
flat = [x for row in matrix for x in row if x % 2 == 0]
print(flat)
print(sum(flat))
```

Answer:

Q80. Predict the output for first-duplicate detection.

Code Snippet

```
nums = [8, 4, 6, 5, 6, 7, 5]
seen = set()
first_dup = None
for x in nums:
    if x in seen:
        first_dup = x
        break
    seen.add(x)
print(first_dup)
```

Answer:

Q81. Predict the output for sorting dict items by value.

Code Snippet

```
d = {'x': 5, 'y': 3, 'z': 4}
out = sorted(d.items(), key=lambda kv: kv[1], reverse=True)
print(out)
```

Answer:

Q82. Predict the output for reversed/sorted behavior and tuple identity slice.

Code Snippet

```
t = (7, 3, 5)
print(tuple(reversed(t)))
print(sorted(t))
print(t[:] is t)
```

Answer:

Q83. Predict the output for dict key collision of bool/int.

Code Snippet

```
d = {}
d[1] = 'int-like'
d[1.0] = 'float-like'
print(d)
print(len(d))
```

Answer:

Q84. Predict the output for function return unpacking and starred target.

Code Snippet

```
def get_data():
    return 7, 7, 10, 9
first, *mid, last = get_data()
print(first)
print(mid)
print(last)
```

Answer:

Q85. Predict the output for dict key collision of bool/int.

Code Snippet

```
d = {}  
d[2] = 'int-like'  
d[2.0] = 'float-like'  
print(d)  
print(len(d))
```

Answer:

Q86. Predict the output for ChainMap lookup order.

Code Snippet

```
from collections import ChainMap  
cfg = ChainMap({'theme': 'light'}, {'theme': 'dark', 'lang': 'en'})  
print(cfg['theme'], cfg['lang'])
```

Answer:

Q87. Predict the output for relation operators on sets.

Code Snippet

```
a = {2, 3}  
b = {2, 3, 4}  
print(a <= b, a < b)  
print(b >= a, b > a)
```

Answer:

Q88. Predict the output for copy behavior with nested tuple data.

Code Snippet

```
import copy  
t1 = (1, [7, 8])  
t2 = copy.copy(t1)  
t3 = copy.deepcopy(t1)  
t1[1].append(13)  
print(t2)  
print(t3)
```

Answer:

Q89. Predict the output for sorting by a tuple key.

Code Snippet

```
items = [('p', 54), ('q', 80), ('r', 66)]
print(sorted(items, key=lambda x: (x[1], x[0])))
```

Answer:

Q90. Predict the output for copy behavior with nested tuple data.

Code Snippet

```
import copy
t1 = (1, [3, 4])
t2 = copy.copy(t1)
t3 = copy.deepcopy(t1)
t1[1].append(9)
print(t2)
print(t3)
```

Answer:

Q91. Predict the output for copy behavior with nested tuple data.

Code Snippet

```
import copy
t1 = (1, [2, 3])
t2 = copy.copy(t1)
t3 = copy.deepcopy(t1)
t1[1].append(8)
print(t2)
print(t3)
```

Answer:

Q92. Predict the output for dict key collision of bool/int.

Code Snippet

```
d = {}
d[4] = 'int-like'
d[4.0] = 'float-like'
print(d)
print(len(d))
```

Answer:

Q93. Predict the output for nested flattening with conditional selection.

Code Snippet

```
matrix = [[1, 7], [1, 7], [4, 8]]
flat = [x for row in matrix for x in row if x % 2 == 0]
print(flat)
print(sum(flat))
```

Answer:

Q94. Predict the output for tuple pattern matching.

Code Snippet

```
point = (0, 11)
match point:
    case (0, y):
        print('Y', y)
    case (x, 0):
        print('X', x)
    case _:
        print('0')
```

Answer:

Part B: MCQ (Single Correct Option)

Q95. ``keys_view = d.keys()`` then adding a key to ``d`` means:

- A) ``keys_view`` stays unchanged
- B) ``keys_view`` updates dynamically
- C) error is raised
- D) ``keys_view`` becomes list

Q96. In expression ``a | b & c``, which operation happens first?

- A) Union ``|``
- B) Intersection ``&``
- C) Left-to-right only
- D) Symmetric difference

Q97. A tuple can be dict key when:

- A) length is 2
- B) all elements are hashable
- C) it contains list
- D) it is sorted

Q98. Best built-in for frequency counting is:

- A) ``itertools.groupby`` only
- B) ``collections.Counter``
- C) ``setdefault`` only
- D) ``heapq``

Q99. ``zip(a, b)`` with different lengths stops at:

- A) Longest iterable
- B) Shortest iterable
- C) Raises `ValueError`
- D) Pads with zeros automatically

Q100. Which operation keeps original list unchanged while sorting with custom key?

- A) ``lst.sort(key=...)``
- B) ``sorted(lst, key=...)``
- C) ``lst.reverse()``
- D) ``reversed(lst)``